

Transformers

Naeemullah Khan

naeemullah.khan@kaust.edu.sa

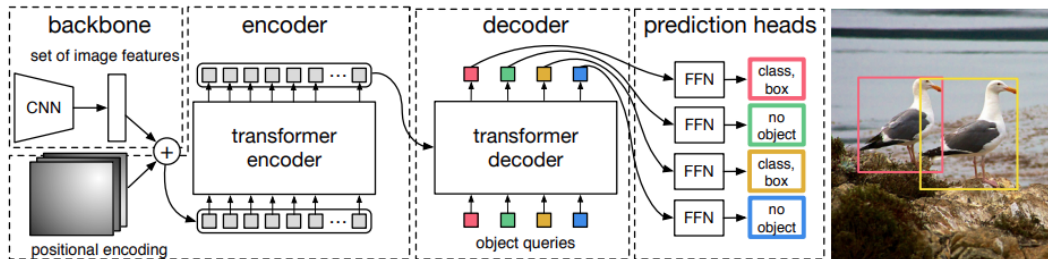


جامعة الملك عبد الله
للعلوم والتقنية

King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

June 20, 2026



Source: [Roboflow — What is DETR?](#) (Carion et al., 2020)

1. Motivation
2. Learning Outcomes
3. General Attention Layer
4. Self Attention Layer
5. Permutation Invariance
6. Transformer Overview
7. RNNs vs Transformers
8. Limitations and Future Directions
9. References

Why this topic matters:

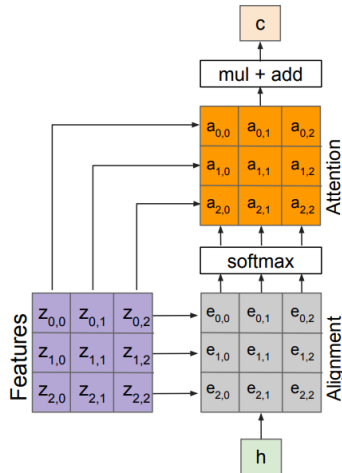
- ▶ RNNs are sequential and struggle with long-range dependencies.
- ▶ Attention lets a model focus on the most relevant parts of the input.
- ▶ Transformers replace recurrence with self-attention — fully parallelizable.
- ▶ They are the backbone of modern NLP and, increasingly, computer vision.
- ▶ *"Attention is all you need."* — Vaswani et al., 2017.

By the end of this session, you will be able to:

- ▶ Explain the general attention layer and the role of queries, keys, and values.
- ▶ Describe self-attention and why it is permutation equivariant.
- ▶ Understand positional encodings, masked attention, and multi-head attention.
- ▶ Describe the Transformer block and how blocks stack into a full Transformer.
- ▶ Compare Transformers with RNNs in terms of parallelism and long-range modeling.

Advanced Computer Vision: **General Attention Layer**

Attention We Just Saw in Image Captioning



Outputs:

context vector: $\mathbf{c}$ (shape: D)

Operations:

Alignment: $e_{i,j} = f_{\text{att}}(\mathbf{h}, \mathbf{z}_{i,j})$

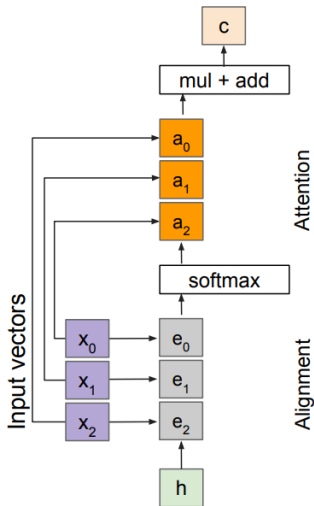
Attention: $\mathbf{a} = \text{softmax}(\mathbf{e})$

Output: $\mathbf{c} = \sum_{i,j} a_{i,j} \mathbf{z}_{i,j}$

Inputs:

Features: $\mathbf{z}$ (shape: $H \times W \times D$)

Query: $\mathbf{h}$ (shape: D)



Outputs:

context vector: $\mathbf{c}$ (shape: D)

Operations:

Alignment: $e_i = f_{\text{att}}(h, x_i)$

Attention: $\mathbf{a} = \text{softmax}(\mathbf{e})$

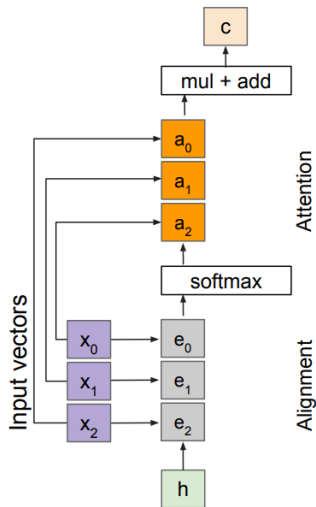
Output: $\mathbf{c} = \sum_i a_i x_i$

- ▶ The attention operation is permutation invariant.
- ▶ It does not depend on the ordering of the features.
- ▶ Stretch $H \times W = N$ into N vectors.

Inputs:

Input vectors: $\mathbf{x}$ (shape: $N \times D$)

Query: $\mathbf{h}$ (shape: D)



Outputs:
context vector: c (shape: D)

Operations:

Alignment: $e_i = h \cdot x_i / \sqrt{D}$

Attention: $a = \text{softmax}(e)$

Output: $c = \sum_i a_i x_i$

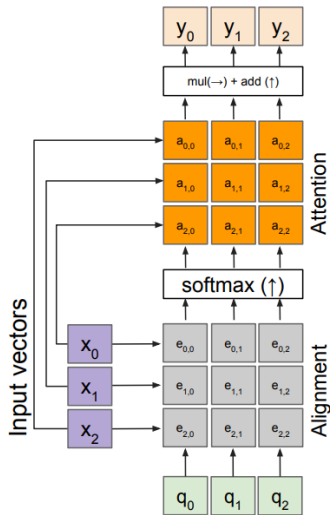
Inputs:

Input vectors: x (shape: $N \times D$)

Query: h (shape: D)

Change $f_{att}(\cdot)$ to a scaled simple dot product:

- ▶ Larger dimensions mean more terms in the dot product sum.
- ▶ Therefore, the variance of the logits is higher. Large-magnitude vectors will produce much higher logits.
- ▶ As a result, the post-softmax distribution has lower entropy, assuming logits are IID.
- ▶ Ultimately, these large-magnitude vectors will cause softmax to peak and assign very little weight to all others.
- ▶ Divide by $\sqrt{D}$ to reduce the effect of large-magnitude vectors.



Outputs:

context vectors: $\mathbf{y}$ (shape: D)

Operations:

Alignment: $e_{i,j} = q_j \cdot x_i / \sqrt{D}$

Attention: $\mathbf{a} = \text{softmax}(\mathbf{e})$

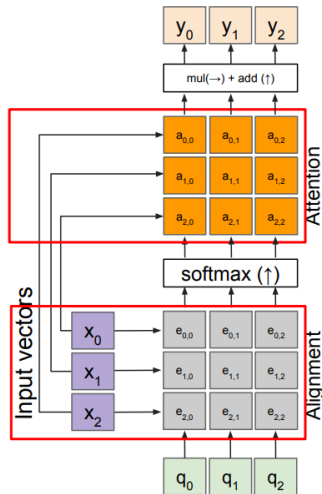
Output: $y_j = \sum_i a_{i,j} x_i$

- We can have multiple query vectors.
- Each query creates a new output context vector.

Inputs:

Input vectors: $\mathbf{x}$ (shape: $N \times D$)

Queries: $\mathbf{q}$ (shape: $M \times D$)

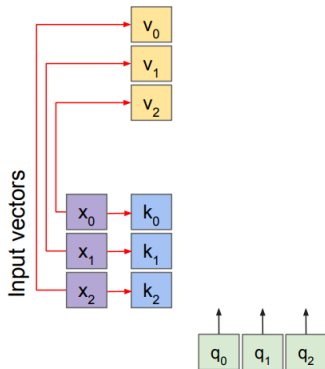


Outputs:
context vectors: $\mathbf{y}$ (shape: D)

Operations:
Alignment: $e_{i,j} = q_j \cdot x_i / \sqrt{D}$
Attention: $\mathbf{a} = \text{softmax}(\mathbf{e})$
Output: $y_j = \sum_i a_{i,j} x_i$

- Notice that the input vectors are used for both the alignment and the attention calculations.

Inputs:
Input vectors: $\mathbf{x}$ (shape: $N \times D$)
Queries: $\mathbf{q}$ (shape: $M \times D$)



Operations:

Key vectors: $\mathbf{k} = \mathbf{x}\mathbf{W}_k$

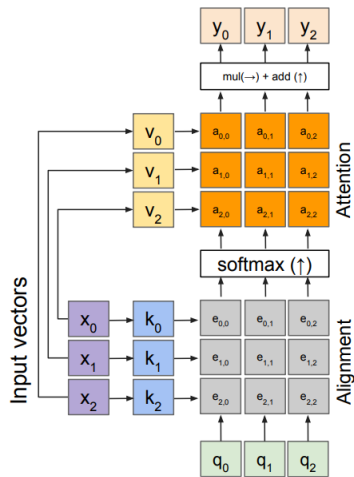
Value vectors: $\mathbf{v} = \mathbf{x}\mathbf{W}_v$

Inputs:

Input vectors: $\mathbf{x}$ (shape: $N \times D$)

Queries: $\mathbf{q}$ (shape: $M \times D_k$)

- Notice that the input vectors are used for both the alignment and the attention calculations.
- We can add more expressivity to the layer by adding a different fully connected (FC) layer before each of the two steps.



Outputs:

context vectors: y (shape: D_v)

Operations:

Key vectors: $k = xW_k$

Value vectors: $v = xW_v$

Alignment: $e_{i,j} = q_i \cdot k_j / \sqrt{D}$

Attention: $a = \text{softmax}(e)$

Output: $y_j = \sum_i a_{i,j} v_i$

Inputs:

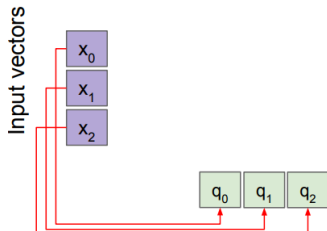
Input vectors: x (shape: $N \times D$)

Queries: q (shape: $M \times D_k$)

- Notice that the input vectors are used for both the alignment and the attention calculations.
- We can add more expressivity to the layer by adding a different fully connected (FC) layer before each of the two steps.
- The input and output dimensions can now change depending on the key and value FC layers.

Advanced Computer Vision: **Self Attention Layer**

- ▶ **Self-Attention:** Computes attention within a single sequence.
- ▶ **How it works:**
 - Each token attends to every other token.
 - Captures global context, not just local dependencies.



Operations:

Key vectors: $\mathbf{k} = \mathbf{x}\mathbf{W}_k$

Value vectors: $\mathbf{v} = \mathbf{x}\mathbf{W}_v$

Query vectors: $\mathbf{q} = \mathbf{x}\mathbf{W}_q$

Alignment: $e_{i,j} = \mathbf{q}_i \cdot \mathbf{k}_j / \sqrt{D}$

Attention: $\mathbf{a} = \text{softmax}(\mathbf{e})$

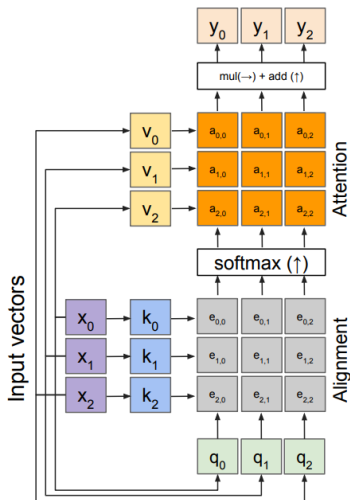
Output: $\mathbf{y}_j = \sum_i a_{i,j} \mathbf{v}_i$

Inputs:

Input vectors: $\mathbf{x}$ (shape: $N \times D$)

Queries: $\mathbf{q}$ (shape: $M \times D_q$)

- Recall that the query vector is a function of the input vectors.
- We can calculate the query vectors from the input vectors, thus defining a "self-attention" layer.
- There are no input query vectors anymore.
- Instead, query vectors are calculated using a fully connected (FC) layer.



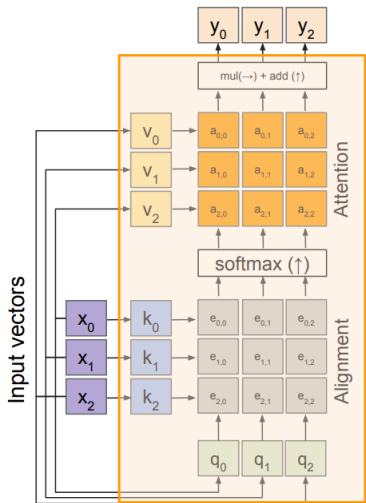
Outputs:
context vectors: $\mathbf{y}$ (shape: D_v)

Operations:
Key vectors: $\mathbf{k} = \mathbf{x}\mathbf{W}_k$
Value vectors: $\mathbf{v} = \mathbf{x}\mathbf{W}_v$
Query vectors: $\mathbf{q} = \mathbf{x}\mathbf{W}_q$
Alignment: $e_{i,j} = q_i \cdot k_j / \sqrt{D}$
Attention: $\mathbf{a} = \text{softmax}(\mathbf{e})$
Output: $y_j = \sum_i a_{i,j} v_i$

Inputs:
Input vectors: $\mathbf{x}$ (shape: $N \times D$)

- Recall that the query vector is a function of the input vectors.
- We can calculate the query vectors from the input vectors, thus defining a "self-attention" layer.
- There are no input query vectors anymore.
- Instead, query vectors are calculated using a fully connected (FC) layer.

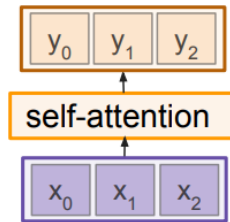
Self Attention Layer



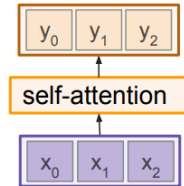
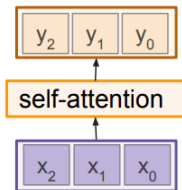
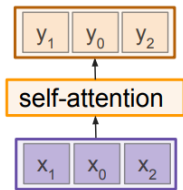
Outputs:
context vectors: $\mathbf{y}$ (shape: D_v)

Operations:
Key vectors: $\mathbf{k} = \mathbf{x}\mathbf{W}_k$
Value vectors: $\mathbf{v} = \mathbf{x}\mathbf{W}_v$
Query vectors: $\mathbf{q} = \mathbf{x}\mathbf{W}_q$
Alignment: $e_{i,j} = q_i \cdot k_j / \sqrt{D}$
Attention: $\mathbf{a} = \text{softmax}(\mathbf{e})$
Output: $y_j = \sum_i a_{i,j} v_i$

Inputs:
Input vectors: $\mathbf{x}$ (shape: $N \times D$)

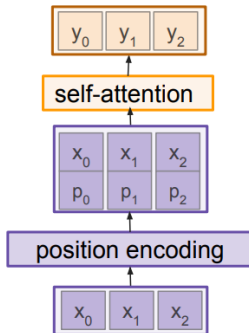


Advanced Computer Vision: **Permutation Invariance**

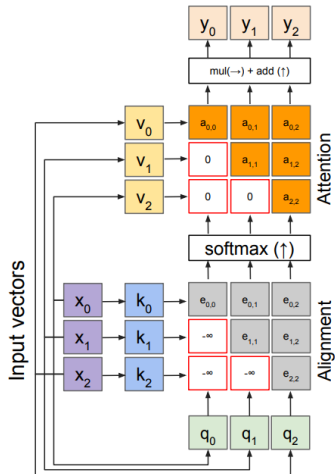


- ▶ Self-Attention Layer is Permutation equivariant
- ▶ It doesn't care about the orders of the inputs!
- ▶ **Problem:** How can we encode ordered sequences like language or spatially ordered image features?

- ▶ Concatenate/add special positional encoding p_j to each input vector x_j
- ▶ We use a function $\text{pos}: N \rightarrow \mathbb{R}^D$ to process the position j of the vector into a d -dimensional vector
- ▶ So, $p_j = \text{pos}(j)$



Masked self-attention layer



Outputs:

context vectors: $\mathbf{y}$ (shape: D_v)

Operations:

Key vectors: $\mathbf{k} = \mathbf{x}\mathbf{W}_k$

Value vectors: $\mathbf{v} = \mathbf{x}\mathbf{W}_v$

Query vectors: $\mathbf{q} = \mathbf{x}\mathbf{W}_q$

Alignment: $e_{i,j} = \mathbf{q}_i \cdot \mathbf{k}_j / \sqrt{D}$

Attention: $\mathbf{a} = \text{softmax}(\mathbf{e})$

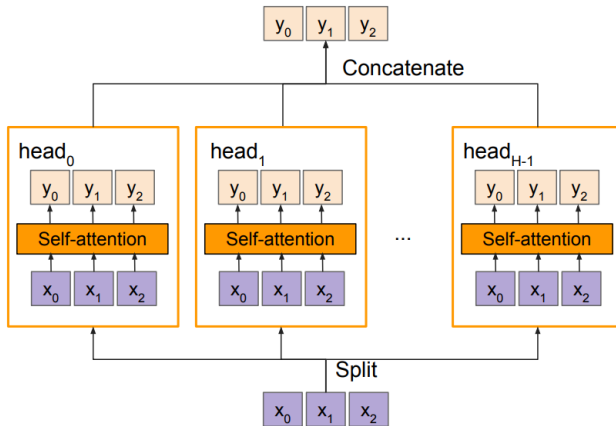
Output: $y_j = \sum_i a_{i,j} v_i$

Inputs:

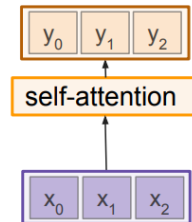
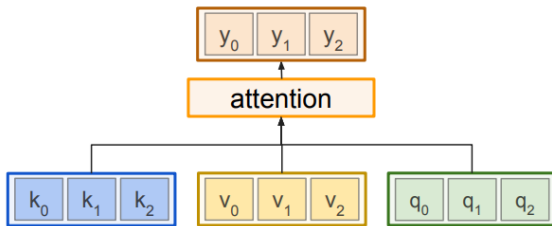
Input vectors: $\mathbf{x}$ (shape: $N \times D$)

- ▶ Prevent vectors from looking at future vectors.
- ▶ Manually set alignment scores to $-\infty$

- Multiple self-attention heads in parallel



General attention versus self-attention



Example: CNN with Self-Attention

Input Image



[Cat image](#) is free to use under the [Pixabay License](#)

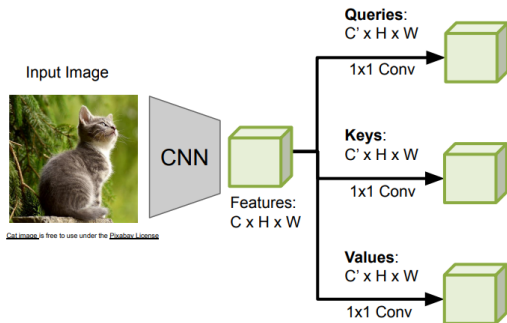


CNN

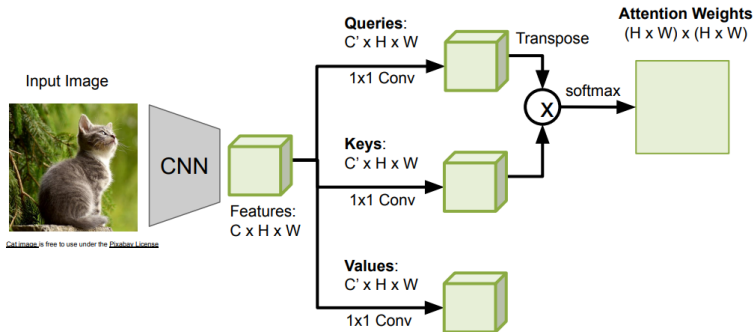


Features:
 $C \times H \times W$

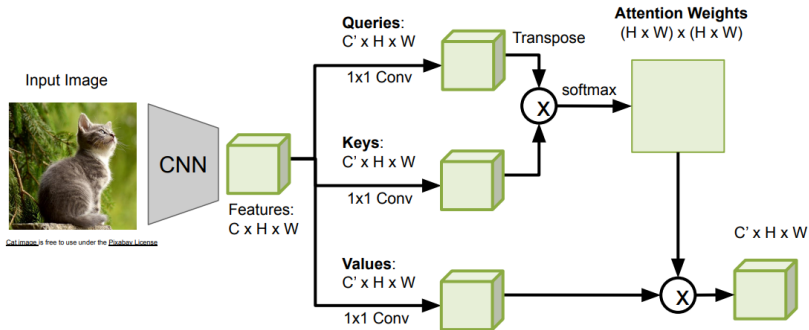
Example: CNN with Self-Attention



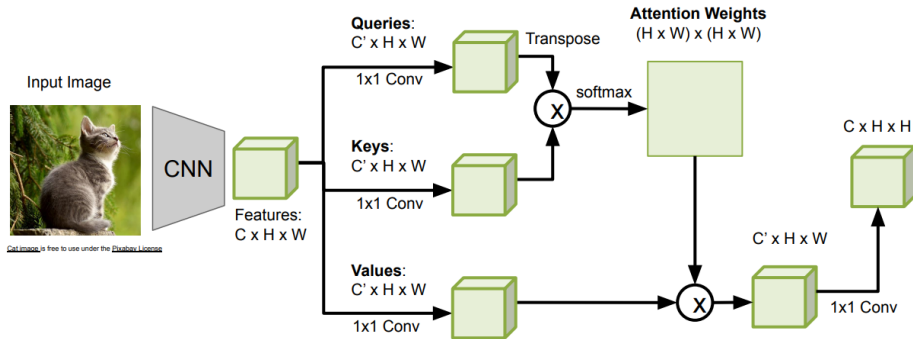
Example: CNN with Self-Attention



Example: CNN with Self-Attention

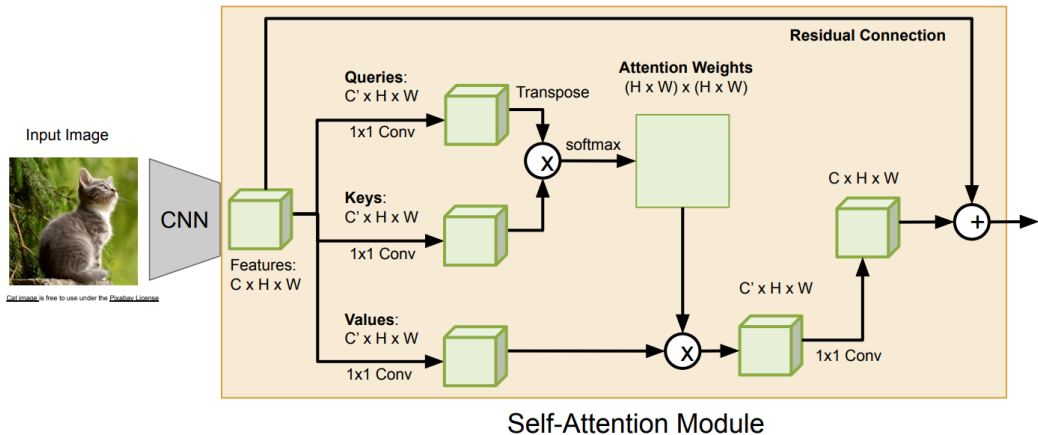


Example: CNN with Self-Attention



Cat image is free to use under the Pixabay License

Example: CNN with Self-Attention

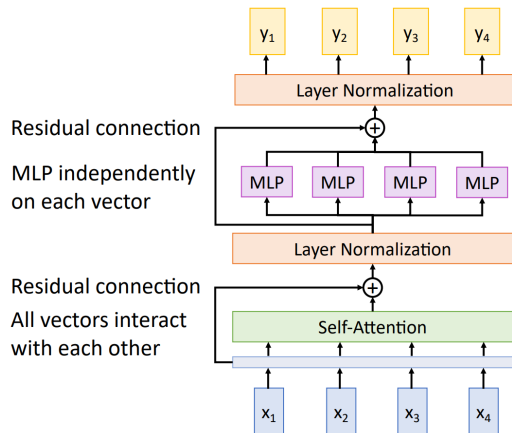


Advanced Computer Vision: **Transformer Overview**

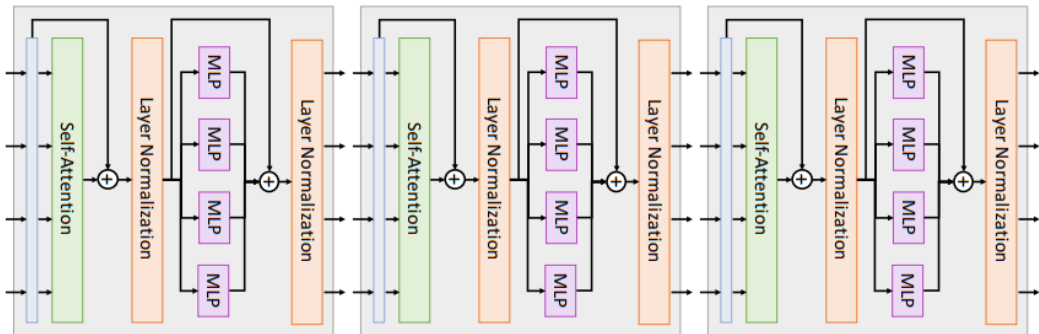
Attention is all you need

Vaswani et al, NeurIPS 2017

- ▶ **Input:** Set of vectors $\mathbf{x}$
- ▶ **Output:** Set of vectors $\mathbf{y}$
- ▶ Self-attention is the only interaction between vectors!
- ▶ Layer normalization and MLP operate independently on each vector.
- ▶ Highly scalable and highly parallelizable.



- ▶ A Transformer consists of a sequence of transformer blocks.
- ▶ Vaswani et al. used 12 blocks, $D_Q = 512$, and 6 attention heads.



Transformers: **RNNs vs Transformers**

Aspect	RNN / LSTM	Transformer
Computation	Sequential	Parallel
Long-range dependencies	Hard (signal decays)	Direct (any-to-any)
Path length between tokens	$O(n)$	$O(1)$
Complexity per layer	$O(n \cdot d^2)$	$O(n^2 \cdot d)$
Order information	Built-in (recurrence)	Added (positional encoding)

- ▶ Transformers trade quadratic attention cost for full parallelism and shorter gradient paths.
- ▶ This makes them far more scalable on modern hardware — the key to large pretrained models.

Transformers: **Limitations and Future Directions**

- ▶ **Quadratic complexity:** self-attention scales as $O(n^2)$ in sequence length.
- ▶ **Data hungry:** need large datasets or pretraining to perform well.
- ▶ **No built-in inductive bias:** must learn structure (e.g., order, locality) from data.
- ▶ **Memory intensive:** long sequences are expensive to store and compute.

- ▶ **Efficient attention:** [Linformer](#), [Performer](#), [Longformer](#), [FlashAttention](#).
- ▶ **Vision:** extend self-attention to images (next topic: [Vision Transformers](#)).
- ▶ **Multimodality:** unify text, image, and audio in a single architecture (e.g., [Flamingo](#)).

Transformers: **References**

- [1] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is All You Need. *Advances in Neural Information Processing Systems*, 30. <https://arxiv.org/abs/1706.03762>
- [2] Bahdanau, D., Cho, K., & Bengio, Y. (2015). Neural Machine Translation by Jointly Learning to Align and Translate. *International Conference on Learning Representations*. <https://arxiv.org/abs/1409.0473>
- [3] Fei-Fei Li, Johnson, J., & Yeung, S. Stanford CS231n: Deep Learning for Computer Vision. <http://cs231n.stanford.edu/>
- [4] Johnson, J. UMich EECS 498.008/598.008: Deep Learning for Computer Vision. <https://web.eecs.umich.edu/~justincj/teaching/eecs498/WI2023/>